Accent Adaptation in Speech Recognition

Nitish Joshi
RnD Report
160070017

Abstract

In this project, we have worked on the problem of accent adaptation in end-to-end speech recognition systems adopting various techniques from the field of domain adaptation. We incorporate domain adversarial training of neural networks in the more general setting where we have transcribed speech from multiple source domains and unlabelled speech samples from one target domain. We perform experiments on the newly released Mozilla Common Voice dataset.

1 Introduction

The success of machine learning based systems is majorly attributed to the enormous amount of datasets that are available nowadays. This is specifically true in the case of end-to-end speech recognition systems which have been known to perform better than classical ASR systems when huge amounts of data are available. Unfortunately, in case of specific problems such as accent adaptation, it is very difficult to expect the availability of English speech transcribed datasets for all the accents.

We explore the multiple-source domain adaptation problem for adapting to accents where we have labelled samples from multiple source accents and unlabelled samples from one target accent. We make use of the newly released Mozilla Common Voice dataset.

The basic ASR model we use is very similar to Listen-Attend-Spell model (Chan et al., 2016). The architecture consists of a pyramidal Long Short Term Memory RNN (LSTM) based encoder and another LSTM based decoder which uses attention over the inputs. This helps the decoder to only focus on specific parts of the model in the decoding process. For the domain adaptation part, the architecture we use is along the lines of what is described in this paper (Zhao et al., 2017). We basically introduce a set of domain classifiers which are able to distinguish the source samples from the target samples and use them in combination with above described LAS based ASR system.

2 Related Work

There have been various theoretical models which have been put forth on Domain Adaptation (Cortes and Mohri, 2014) ; (Ben-David et al., 2007). The domain adaptation techniques used in this project are majorly backed by the second one and its extension (Ben-David et al., 2010).

The technique of domain adversarial neural networks has turned out to be successful in the area of computer vision and was first explored for the single source domain setting (Ganin et al., 2016). It was recently extended for the multiple source domain setting (Zhao et al., 2017).

In the field of Speech Recognition, domain adversarial training for accent adaptation was most recently explored in (Sun et al., 2017), but it is restricted to single source - target setting for Mandarin Speech.

3 End-to-end ASR Systems

3.1 Connectionist Temporal Classification

One of the major issues in training of speech recognition systems is the alignment required between audio and transcription sequences. Moreover, the exact alignment is irrelevant in most speech tasks where only the word level transcriptions matter. Connectionist Temporal Classification (CTC) is an objective function that helps address this problem and requires no prior alignment between the input and output sequences (Graves and Jaitly, 2014).

Apart from the transcription labels, the output layer has another label known as the 'blank' label which corresponds to a null emission. Thus the softmax function over the final outputs at a time step will give us a probability distribution over all the output characters including the 'blank' label. The probability of a sequence s is the product over emission probabilities at each time step :

$$P(s|x) = \prod_{t=1}^{T} P(s_t, t|x) \tag{1}$$

CTC then uses a many-to-one mapping(F) between the alignments obtained from the model (s) and the transcription sequence (y) by first removing the consecutive occurrences of the same label and then removing all the occurrences of 'blank' label from the output sequence.

$$P(y|x) = \sum_{s \in F^{-1}(y)} P(s|x) \tag{2}$$

Thus by summing over all places where the labels could occur, we avoid the problem of prealignment of input and output sequences.

3.2 Attention-based framework

This method usually relies on sequence to sequence learning frameworks which are useful in the case of variable length inputs and outputs. The model basically consists of an encoder network (which are often RNN based) which maps the variable length input to a fixed length vector. It then uses a similar decoder to output the variable length output emitting one label at every time step.

The attention-based frameworks use an attention over the input sequence at every time step of the decoder so that the decoder can only focus on specific parts of the input. This in turn relaxes the condition that the input to the decoder is a fixed-length vector and allows it pick up only some selective parts from the input (Chan et al., 2016).

Specifically, if $\mathbf{h}$ is the encoded vector for a given input sample, then the context vector $\mathbf{c_i}$ for ith decoding step is computed as:

$$c_i = \sum_j \alpha_{i,j} h_j \tag{3}$$

where the weights $\alpha_{i,j}$ are generated as:

$$\alpha_{i,j} = \frac{exp(e_{i,j})}{\sum_k exp(e_{i,k})} \tag{4}$$

$$e_{i,j} = a(s_i, h_j) \tag{5}$$

where a is a matching alignment between the previous state of the decoder s_i and jth component of the encoded input h_j (Bahdanau et al., 2014).

4 Theoretical Analysis of Domain Adaptation

In this section, we discus the theoretical model of domain adaptation put forward in (Ben-David et al., 2007) and (Blitzer et al., 2008). It basically formalizes the problem of domain adaptation and gives a bound on the classification error on the target domain for models trained on a different source domains.

A domain is defined by a distribution $\mathcal{D}$ over the instance set $\mathcal{X}$ and we associate with it a labelling function $f : \mathcal{X} \rightarrow [0, 1]$ (The analysis is restricted to binary classification for simplicity). The hypothesis function is defined as $h : \mathcal{X} \rightarrow \{0, 1\}$ and the risk of a hypothesis function is defined as:

$$\epsilon_s(h) = E_{x \sim \mathcal{D}_s}[|h(x) - f(x)|] \tag{6}$$

Here the subscript s refers to the source domain and we stick to the case of one source and one target domain. To measure the distance between two distributions the idea of $\mathcal{H}$-divergence is used. Specifically, if $\mathcal{H}$ is a hypothesis class over $\mathcal{X}$, then the distance between two distributions $\mathcal{D}$ and $\mathcal{D}^{'}$ is defined as :

$$d_{\mathcal{H}}(\mathcal{D}, \mathcal{D}^{'}) = 2 \sup_{\mathcal{A} \in \mathcal{A}_{\mathcal{H}}} |Pr_D(A) - Pr_{D'}(A)| \tag{7}$$

where $\mathcal{A}_{\mathcal{H}} : \{h^{-1}(\{1\}) | h \in \mathcal{H}\}$ is the defined set of subsets of $\mathcal{X}$ that are support of some hypothesis h in $\mathcal{H}$. Next we define the symmetric difference hypothesis class:

$$\mathcal{H}\Delta\mathcal{H} = \{h(\mathbf{x}) \oplus h^{'}(\mathbf{x}) : h, h^{'} \in \mathcal{H}\} \tag{8}$$

This class thus contains all those hypothesis g such that it is 1 whenever some two hypothesis in the original class disagree. We define the set $A_{\mathcal{H}\Delta\mathcal{H}}$ as above. The final quantity defined is the minimum risk (or error) of the ideal hypothesis function in $\mathcal{H}$. The ideal hypothesis function should minimize the combined source and target risk.

$$h^* = argmin_{h \in \mathcal{H}} \epsilon_s(h) + \epsilon_t(h) \tag{9}$$

$$\lambda = \epsilon_s(h^*) + \epsilon_t(h^*) \tag{10}$$

The main theorem which is proved in the paper and which puts a bound on the error on target domain is the following:

Theorem1 Let $\mathcal{H}$ be a hypothesis space of VC dimension d and $\mathcal{U}_s, \mathcal{U}_t$ be unlabelled samples from the source and target domain with distributions $\mathcal{D}_s$ and $\mathcal{D}_t$ respectively each of size $m^{'}$. Let $\hat{d}_{\mathcal{H}\Delta\mathcal{H}}$ be the empirical distance on $\mathcal{U}_s, \mathcal{U}_t$ induced by the symmetric difference hypothesis space. With probability of atleast $1-\delta$ (over the choice of samples), for every $h \in \mathcal{H}$:

$$\epsilon_t(h) \leq \epsilon_s(h) + \frac{1}{2}\hat{d}_{\mathcal{H}\Delta\mathcal{H}}(\mathcal{U}_s, \mathcal{U}_t) + 4\sqrt{\frac{2d\log(2m') + \log(\frac{4}{\delta}}{m^{'}}} + \lambda \tag{11}$$

The first term on the right side tells that the error on the target domain is bounded by the empirical training error on the source domain. The second term is the distance between the source and target domain distributions. This thus tells that the model should try to simultaneously minimize the source training error as well as learn a good representation of the two domains to minimize the distance between them. The last term λ is the minimum combined error that can be achieved by a hypothesis in $\mathcal{H}$. Thus we cannot hope for a good target hypothesis by training only on the source when λ itself is large. But for small values of λ, the first two terms are important quantities for the bound on target error.

4.1 Extension to Multiple Source Domains

The recent paper (Zhao et al., 2018) on multiple source domain adaptation extends the theoretical model described in the above section.

Firstly, the definition of $\mathcal{H}$-distance has to be extended for the case of multiple source domains. Let $\{\mathcal{D}_{S_i}\}_{i=1}^k$ and $\mathcal{D}_\mathcal{T}$ be the k source domains and target domain respectively. The distance function is then defined as :

$$d_\mathcal{H}(\mathcal{D}_\mathcal{T}, \{\mathcal{D}_{S_i}\}_{i=1}^k) = max_{i\in[k]} d_\mathcal{H}(\mathcal{D}_\mathcal{T}, \mathcal{D}_{S_\rangle})$$

where $d_\mathcal{H}()$ is as defined in the previous section.

The first theorem bounds the true risk on target domain with the discrepancy measure and true risk on the source domain.

$$\epsilon_t(h) \leq \max_{i\in[k]} \epsilon_{s_i}(h) + \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(\mathcal{D}_\mathcal{T}, \{\mathcal{D}_{S_i}\}_{i=1}^k) + \lambda \tag{12}$$

The next set of theorems then bound the true risk on the source domain and the discrepancy distance $d_{\mathcal{H}\Delta\mathcal{H}}(\mathcal{D}_\mathcal{T}, \{\mathcal{D}_{S_i}\}_{i=1}^k)$ by their corresponding empirical estimates since the prior are usually unknown and the empirical estimates can be computed from $i.i.d$ samples from the target and source distributions. The proofs of these theorems use results from VC-theory of reduction from infinite hypothesis space to finite space when working with finite samples. Combining them with the first theorem gives the following final theorem:

Theorem2 Let $\mathcal{D}_\mathcal{T}$ and $\{\mathcal{D}_{S_i}\}_{i=1}^k$ be the target and k source distributions over $\mathcal{X}$. Let $\mathcal{H}$ be a hypothesis class with VCdim($\mathcal{H}$) = d. If $\widehat{\mathcal{D}_\mathcal{T}}$ and $\left\{\widehat{\mathcal{D}_{S_i}}\right\}_{i=1}^k$ are the empirical distributions of $\mathcal{D}_\mathcal{T}$ and $\{\mathcal{D}_{S_i}\}_{i=1}^k$ generate by m $i.i.d$ samples from each domain, then for $0 < \delta < 1$ with probability atleast $1-\delta$ (over the choice of samples), we have:

$$\epsilon_t \leq \max_{i\in[k]} \hat{\epsilon}_{s_i}(h) + \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(\hat{\mathcal{D}}_T, \left\{\hat{\mathcal{D}}_{S_i}\right\}_{i=1}^k) + \mathcal{O}\left(\sqrt{\frac{1}{m}\left(\log(\frac{k}{\delta}) + d\log(\frac{me}{d})\right)}\right) + \lambda \tag{13}$$

The first term on the right is the worst case error of some hypothesis h in hypothesis class $\mathcal{H}$ for the k source domains. The second term is the distance between the target domain and the k source domains by the distance measure defined previously. The last term similar to the single source setting is the optimal error of the task which should be small if we hope for low error on the target error by training on the source domain. It is easy to see that this reduces to the single source bound of the previous section when $k = 1$.

5 Multisource Domain Adversarial Networks

This section describes the Multisource Domain Adversarial Networks (MDAN) for ASR systems

which were introduced in (Zhao et al., 2018) and are extension of the Domain Adversarial Networks which were introduced in (Ganin et al., 2016).

5.1 Model

Suppose we have m samples with transcription from each of the k source domains and additionally we have m samples from the target domain for which we have no transcription available. The main idea behind MDANs is to train a network that has the two following properties :

1. The features learnt by the feature extractor should be indistinguishable for the k source domains and the target domain. This will the ensure that the features have no domain specific information and are thus able to capture the domain invariant features from speech samples.

2. The feature representations obtained should contain enough information for our main task of speech recognition to succeed.

Note that without the second requirement, we could very well learn some random noise as our representation which is indeed indistinguishable across all the domains. Similarly without the first requirement, our features learnt may be very specific to a particular domain and will not be able to generalize across all the domains.

The model consists of the three following types of networks : 1. Feature Extractor : $\mathcal{F}()$ 2. Desired Task Network : $\mathcal{G}()$ which in our case is ASR 3. k Domain Classifiers : $\mathcal{S}_i()$ for classification of samples from the ith source domain and target domain.

The feature extractor $\mathcal{F}$ takes as input $\mathbf{x}$ which is the sequence of filter bank spectra features and produces $\mathbf{h} = \mathcal{F}(\mathbf{x})$ which is the encoded representation of the input. We use $\mathcal{F}$ as described in (Chan et al., 2016). It basically consists of multi-stacked Bidirectional Long Short Term Memory RNN (BLSTM) with a pyramidal shape : After every layer, the time resolution factor is reduced to half i.e. outputs at consecutive time steps are concatenated before feeding to the next layer. This thus reduces the size of the vector h and captures the information contained in the long vector x compactly. Similar to (Chan et al., 2016), we use 3 layers thus reducing the time resolution factor by 8.

The network $\mathcal{G}$ takes as input the encoded input h and produces a probability distribution over character sequences. Again similar to LAS model, $\mathcal{G}$ is attention-based LSTM transducer. At every time step, it produces a probability distribution over the next character conditioned on the previous characters with the input being the decoder state and the context vector, which is produced by the attention mechanism as described in (Bahdanau et al., 2014). Thus $\mathcal{F}$ and $\mathcal{G}$ together describe the model for a traditional attention base44926d ASR system.

The domain classifier $\mathcal{S}_i$ takes as input the encoded input h from either the ith source domain or the target domain and produces a binary label identifying from which of the two domains the input belongs. The model we use for this binary classification task consists of a two layered BLSTM followed by a simple MLP consisting of one hidden layer.

5.2 Training

To minimize the bound obtained in (13), the first two terms on the right side are of interest since the other two are decided when we fix the hypothesis class $\mathcal{H}$.

$$\text{minimize} \max_{i \in [k]} \left(\hat{\epsilon}_{s_i}(h) + \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(\hat{\mathcal{D}_T}, \left\{ \hat{\mathcal{D}}_{S_i} \right\}_{i=1}^{k}) \right) \tag{14}$$

As shown in (Ben-David et al., 2007), computing the distance measure is relating to learning a domain classifier thus giving the following simplification :

$$\text{minimize} \max_{i \in [k]} \left(\hat{\epsilon}_{s_i}(h) - \min_{h' \in \mathcal{H}\Delta\mathcal{H}} \hat{\epsilon}_{T,S_i}(h') \right) \tag{15}$$

where $\hat{\epsilon}_{T,S_i}(h)$ is the empirical risk of h in the domain discrimination task. We can directly implement the minimization problem of (15) using gradient reversal layer(GRL) as in (Ganin et al., 2016). Specifically by introducing the GRL between the feature extractor $\mathcal{F}$ and the domain classifiers $\mathcal{S}_i$ while backpropogating through the GRL, we multiply the gradients by a parameter $\lambda < 0$ before we change the parameters of the network $\mathcal{F}$ with respect to the loss of domain classification task. In the forward pass, the GRL simply acts as

an identity function making no changes to the input. Thus, we let the parameters of $\mathcal{F}$ change to maximize the error of domain classification while the parameters of $\mathcal{S}_i$ to minimize the loss. Intuitively, this tells that the parameters which $\mathcal{F}$ will learn should make the domain classification task harder establishing the idea of domain invariant features.

The direct implementation of (15) will be data inefficient since in every iteration the parameters of the feature extractor $\mathcal{F}$ will only be updated from gradients based on one of the k domains. To make this update process more efficient, we approximate the max function using the log-sum-exp function giving the following optimization task:

$$\text{minimize } \frac{1}{\gamma} \log \sum_{i \in [k]} \left(\hat{\epsilon}_{s_i}(h) - \min_{h' \in \mathcal{H}\Delta\mathcal{H}} \hat{\epsilon}_{T,S_i}(h') \right) \tag{16}$$

where $\gamma > 0$ is a parameter that controls the approximation. Thus (16) provides an efficient updating scheme where we combine the loss from all the k source domains and can then use backpropogation adaptively using a gradient reversal layer as described before.

We use a cross entropy loss for the domain classification task and a sequence to sequence loss for the transcription loss as in the LAS model. For decoding, we use a beam search algorithm, where at every step of the decoding we keep only the β most likely beams. We also incorporate a language model during the process if decoding as described in the next section.

5.3 Language Model Rescoring

The end-to-end ASR systems usually lack in capturing all the linguistic knowledge and it has been seen that the performance of these systems can be boosted by incorporating an external language model (LM) which can help to capture the properties of language.

Specifically, a n-gram LM captures the probability of a certain word occurring conditioned on the $n-1$ previous words. If a string s consists of the words $w1...wl$, then the probability of the string s is:

$$p(s) = \prod_{i=1}^{l+1} p(w_i | w_{i-n+1}^{i-1}) \tag{17}$$

where the w_i with indices less than 1 are taken to be BOS token (beginning of sentence) and w_{l+1} is taken to be the EOS token (end of sentence).

In ASR systems, we rescore the probabilities obtained from our decoder in beam search using the LM scores. Specifically, if s denotes the scores by the model (over all the words) and the LM scores are denoted by s_{lm}, then the final rescored log probabilities over all the words is given by :

$$s_{final} = L(s) + \lambda * L(s_{lm}) \tag{18}$$

where L(.) is the log soft max function computing the respective log probabilities and λ is the weight given to the language model scores. We then use this rescored probabilities along each beam in the beam search decoding process.

6 Results

Here we report the baseline results using the end-to-end model as described above on the Mozilla Common Voice dataset. The model is directly trained on speech samples from four different domains without any domain adaptation and we test it on a different target domain.

We have used all the accent annotated data from Mozilla Common Voice corresponding to the following four accents as source domains with the mentioned number of samples from each domain : US accent : 30877, British (UK) accent : 14851, Australian accent : 4271 and Canadian accent : 3880. We test the model on 300 NZ accent samples and have kept aside the others for domain adaptation.

We experimented with the LM weight as well as the dropout rate and found these to be the best set of parameters (with respect to CER) : LM weight $\alpha = 0.1$ and keep-probability $k = 0.8$ for dropout. We present the results for various values of the LM rescoring weight in 1. Using the above hyperparameters we get a CER of 19.69 and WER of 27.93.

7 Multinomial Adversarial Networks

This section describes a recently introduced model for multi domain text classification (Chen and Cardie, 2018) which achieves results comparable to MDANs presented in the previous section. Additionally, the multinomial adversarial networks are more general in the sense that they can be used in the case of multiple target domains which the MDANs have to treat as independent problems.

LM Weight	WER	CER
0.3	**27.33**	20.26
0.2	27.62	20.25
0.1	27.93	**19.69**
0.05	29.61	20.43

Table 1: Comparison of the Word Error Rate (WER) and Character Error Rate (CER) by varying the Language Model weight α. We get the best CER of 19.69 at $\alpha = 0.1$ and best WER of 27.33 for $\alpha = 0.3$.

7.1 Model

Suppose we have N_1 labeled domains (Δ_L) and N_2 target domains which are unlabeled (Δ_U). Let $\Delta = \Delta_L \cup \Delta_U$ denote the collection of all domains and $N = N_1 + N_2$ represent the total number of domains.

The model essentially consists of four types of networks : A shared feature extractor $\mathcal{F}_s$, a domain feature extractor $\mathcal{F}_{d_i}$ for each labeled domain, the main classifier network $\mathcal{C}$ which in that case is a text classifier and a domain discriminator $\mathcal{D}$.

During the main training iteration, the parameters of $\mathcal{F}_s$, $\mathcal{F}_{d_i}$ and $\mathcal{C}$ are updated using the loss computed from the classifier $\mathcal{C}$ as well as $\mathcal{J}^{\mathcal{D}}_{\mathcal{F}_s}$ which is negatively related to the loss of the domain discriminator $\mathcal{J}_{\mathcal{D}}$. Thus $\mathcal{F}_s$ aims to confuse $\mathcal{D}$ by not allowing it correctly classify the domain. This is based on the idea that if even a good domain discriminator $\mathcal{D}$ cannot predict the domain of the input, then the features learnt by $\mathcal{F}_s$ must be domain invariant. This intuition is very similar to the one which was used in the previously described MDANs.

Since $\mathcal{F}_s$ is forced to learn the domain invariant features, the features extracted by the network $\mathcal{F}_{d_i}$ will be specific to the ith domain and and will be beneficial in the actual task. This differentiates Multinomial Adversarial Networks (MANs) from MDANs, since the prior tries to separately learn the the domain invariant and domain specific features whereas MDANs try to directly learn the domain invariant features. Also, the parameters of the domain discriminator $\mathcal{D}$ are updated separately using samples from all the domains Δ as opposed to the combined training process in MDANs.

7.2 Training

In the main training iteration we draw a mini batch of samples $(\mathbf{x}, \mathbf{y})$ from each $d \in \Delta_L$, and compute the shared and domain specific features as : $\mathbf{f}_s = \mathcal{F}_s(\mathbf{x})$ and $\mathbf{f}_d = \mathcal{F}_d(\mathbf{x})$. We then concatenate $\mathbf{f}_s$ and $\mathbf{f}_d$ and feed it as input to the main classifier $\mathcal{C}$ and compute the $\mathcal{C}$ loss using the label $\mathbf{y}$. Next, we draw a mini-batch $\mathbf{x}$ from each $d \in \Delta$ and compute share features $\mathbf{f}_s = \mathcal{F}_s(\mathbf{x})$ and compute the domain loss $\mathcal{J}^{\mathcal{D}}_{\mathcal{F}_s}$ which is negatively related to the loss of the domain discriminator. We then use the combined $\mathcal{C}$ loss and domain loss to update the parameters of $\mathcal{F}_s$, $\mathcal{F}_{d_i}$ and $\mathcal{C}$.

The parameters of the domain discriminator are updated separately, by drawing a mini-batch $\mathbf{x}$ from each $d \in \Delta$ and computing the domain discrimnator loss $\mathcal{J}_{\mathcal{D}}$ using the features extracted from the network $\mathcal{F}_s$. At the test time for unlabeled domains, the domain features are set to $\mathbf{0}$ for the input to the classifier $\mathcal{C}$

Acknowledgments

I would like to thank Professor Preethi Jyothi for her immense guidance throughout this project. I would also like to thank Vishal Agrawal for his deepshinx codebase https://github.com/vagrawal/deepsphinx which is an implementation of end-to-end attention-based ASR system in Tensorflow.

References

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2014. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473.

Shai Ben-David, John Blitzer, Koby Crammer, Alex Kulesza, Fernando Pereira, and Jennifer Wortman Vaughan. 2010. A theory of learning from different domains. *Machine Learning*, 79(1):151–175.

Shai Ben-David, John Blitzer, Koby Crammer, and Fernando Pereira. 2007. Analysis of representations for domain adaptation. In B. Schölkopf, J. C. Platt, and T. Hoffman, editors, *Advances in Neural Information Processing Systems 19*, pages 137–144. MIT Press.

John Blitzer, Koby Crammer, Alex Kulesza, Fernando Pereira, and Jennifer Wortman. 2008. Learning bounds for domain adaptation. In J. C. Platt, D. Koller, Y. Singer, and S. T. Roweis, editors, *Advances in Neural Information Processing Systems 20*, pages 129–136. Curran Associates, Inc.

William Chan, Navdeep Jaitly, Quoc V. Le, and Oriol Vinyals. 2016. Listen, attend and spell: A neural network for large vocabulary conversational speech recognition. In *ICASSP*.

Xilun Chen and Claire Cardie. 2018. Multinomial adversarial networks for multi-domain text classification. *CoRR*, abs/1802.05694.

Corinna Cortes and Mehryar Mohri. 2014. Domain adaptation and sample bias correction theory and algorithm for regression.

Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario Marchand, and Victor Lempitsky. 2016. Domain-adversarial training of neural networks. *Journal of Machine Learning Research*, 17(59):1–35.

Alex Graves and Navdeep Jaitly. 2014. Towards end-to-end speech recognition with recurrent neural networks. In *Proceedings of the 31st International Conference on Machine Learning*, volume 32 of *Proceedings of Machine Learning Research*, pages 1764–1772, Bejing, China. PMLR.

Sining Sun, Ching-Feng Yeh, Mei-Yuh Hwang, Mari Ostendorf, and Lei Xie. 2017. Domain adversarial training for accented speech recognition.

H. Zhao, S. Zhang, G. Wu, J. P. Costeira, J. M. F. Moura, and G. J. Gordon. 2017. Multiple Source Domain Adaptation with Adversarial Training of Neural Networks. *ArXiv e-prints*.

Han Zhao, Shanghang Zhang, Guanhang Wu, Jo ao P. Costeira, José M. F. Moura, and Geoffrey J. Gordon. 2018. Multiple source domain adaptation with adversarial learning.